

Architecture and Technical Specification

CAutomation / CCore / User & Client Management

Document status: Version 4 Architecture & Technical Specification baseline candidate aligned with the frozen User & Client Management SRS baseline. This version preserves the Iteration 3 technical contracts and explicitly binds the SpecificationGapManifest output path for deterministic AI generation.

1. Architecture Boundary and Traceability Rule

This ATS defines how the approved User & Client Management SRS is implemented. It is an engineering contract, not a source of new business scope. Generated code must map every table, endpoint, service, validation, screen, report, and test back to the SRS requirement, validation, or acceptance identifiers.

- No ATS table, field, endpoint, service, screen, or test may introduce business requirements absent from the frozen SRS.
- If a required SRS-to-ATS mapping is missing, generation must stop and emit a specification gap report.
- Project-level contracts govern shared platform architecture, stack, authentication, authorization, repository rules, and engineering conventions.
- This module owns client, user, invitation, role, client membership, project membership, administrative report, validation, and tenant isolation implementation contracts only.

2. Layered Architecture Contract

Layer	Mandatory responsibility	Must not contain
Frontend	React TypeScript screens for client administration, user administration, invitation management, memberships, roles, audit views, validation errors, and reports.	Database access or authoritative tenant-isolation decisions.
API	FastAPI REST endpoints exposing authenticated module operations scoped by active Client context and explicit resource identifiers.	Frontend rendering logic, direct SQL, or bypasses around service authorization.
Application services	Use-case orchestration, lifecycle transitions, tenant containment checks, validation coordination, audit emission, and transaction boundaries.	Raw request parsing, UI concerns, or direct AI-provider coupling.
Domain model	Typed entities for Client, User, Role, User Invitation, Client Membership, Project Membership, and Administrative Report.	Framework-specific persistence behavior or transport DTO logic.
Repositories	Persistence access and deterministic query encapsulation for PostgreSQL-backed entities.	Business orchestration, permission policy, or lifecycle decisions.
Database	PostgreSQL referential integrity, uniqueness, lookup/status rows, timestamps, archival fields, and audit relations.	Business behavior that exists only as implicit database side effects.
Validation/reporting	Stable validation identifiers, structured error envelopes, immutable administrative reports, and traceable evidence.	Unstructured messages as the only validation evidence.
AI Engine boundary	Consumes this SRS and ATS as contracts and emits generation artifacts only inside approved module scope.	Invented behaviors, external workflow execution, or direct mutation of project-level contracts.

3. Database Schema Contract

The schema must implement global user identity separately from tenant access. A User does not gain tenant visibility by existing; visibility requires active Client Membership and, when project context is involved, active Project Membership within the same Client boundary.

Table	Purpose	Required fields
client	Tenant account boundary.	id, code, legal_name, name, description, client_type_id, client_status_id, client_target_id, owner_user_id, created_at, updated_at, archived_at
client_type	Lookup table for client classification.	id, code, name, description, is_active, sort_order
client_status	Lookup table for client lifecycle state.	id, code, name, description, is_active, sort_order
client_target	Lookup table for client operational target.	id, code, name, description, is_active, sort_order
user_account	Global identity registry.	id, email, display_name, description, user_status_id, registration_type_id, created_at, updated_at, archived_at
user_status	Lookup table for user lifecycle state.	id, code, name, description, is_active, sort_order
registration_type	Lookup table for registration source.	id, code, name, description, is_active, sort_order
role	System-wide role catalog.	id, code, name, description, role_type_id, is_active, created_at, updated_at, archived_at
role_type	Lookup table for role classification.	id, code, name, description, is_active, sort_order
client_membership	Associates a global User with exactly one Client tenant boundary.	id, client_id, user_id, role_id, membership_status_id, created_at, updated_at, archived_at
project_membership	Associates a User with a Project inside an	id, project_id, client_id, user_id, role_id, project_membership_status_id, created_at,

	already valid Client Membership boundary.	updated_at, archived_at
user_invitation	Secure invitation for tenant onboarding.	id, client_id, issuer_user_id, target_role_id, token_hash, invitation_status_id, expires_at, accepted_at, cancelled_at, created_at
administrative_report	Immutable administrative audit and reporting record.	id, report_type, client_id, user_id, source_action, source_requirement_id, source_validation_id, payload_json, generated_at, reviewed_at, archived_at

4. Relational Constraint Contract

Constraint	Implementation rule
Client code uniqueness	client.code must be globally unique and immutable after activation unless a controlled administrative correction is approved.
User email uniqueness	user.account.email must be globally unique and normalized before persistence.
Client Membership uniqueness	client_membership must enforce unique(client_id, user_id) for non-archived active/suspended membership rows.
Project Membership precondition	project_membership cannot be created unless an active client_membership exists for the same client_id and user_id.
Invitation binding	user_invitation.token_hash must be unique and bound to one client_id for its full lifecycle.
Soft archival	Client, User, Role, Client Membership, Project Membership, and Administrative Report records are not hard-deleted through module services.
Lookup integrity	Status/type/target fields must reference active lookup rows before create or activation operations commit.
Audit immutability	administrative_report payload and source references must not be updated after generation; only reviewed_at or archived_at may change through controlled lifecycle operations.

5. Backend Tenant Isolation Contract

- Every tenant-scoped service method must receive authenticated_user_id and active_client_id or derive them from the authenticated session context through a project-level context provider.
- No endpoint may return Client, Project, Membership, Invitation, Role assignment, or report data for a Client unless the requester has an active membership or platform-level authority.
- Project-scoped operations must verify that the requested Project belongs to the requested Client and that the User has a valid Client Membership before Project Membership checks are evaluated.
- Client Suspended or Archived states immediately block downstream module access for non-platform roles.
- Cross-tenant mismatches must fail with SRS-VAL-011 and emit an isolation-breach audit event.

Tenant Collection Scoping Mandate: Tenant context for collection endpoints must be resolved only at the server-side controller layer from the validated authenticated session context token and project-level context provider. Collection routes must not trust client-supplied query strings, unverified route manipulation, or arbitrary request headers as the source of tenant authority. Explicit {client_id} route values, where present, are treated as requested resource identifiers and must be matched against the authenticated active_client_id before service or repository execution proceeds. Cross-tenant mismatch must fail with SRS-VAL-011 and produce audit evidence.

6. REST API Endpoint Contract

All tenant collection endpoints must resolve an active tenant boundary either from an explicit parent client path parameter, a project-level authenticated tenant context provider, or a secure approved tenant-context header. If active_client_id cannot be resolved and the requester does not have platform-level authority, the endpoint must fail with SRS-VAL-011 before repository access. Collection routes must never perform unbounded global database sweeps.

Endpoint	Method	Service	Mandatory scoping rule	SRS trace
/api/user-client/clients	GET/POST	ClientService	GET returns only authorized Clients unless platform authority exists; POST requires platform authority. Tenant scope must be derived from validated session context token; client-supplied query parameters or unverified headers are not authority sources.	SRS-FR-001, SRS-FR-002
/api/user-client/clients/{client_id}	GET/PATCH	ClientService	client_id must match active_client_id or platform authority.	SRS-FR-001, SRS-FR-009
/api/user-client/clients/{client_id}/suspend	POST	ClientLifecycleService	client_id must be authorized; suspension cascade must execute via validated service path.	SRS-FR-009, SRS-VAL-003
/api/user-client/clients/{client_id}/users	GET/POST	UserService	User collection is filtered by client_membership for client_id; global identity does not imply	SRS-FR-003, SRS-FR-007, SRS-FR-012

			visibility. Tenant scope must be derived from validated session context token; client-supplied query parameters or unverified headers are not authority sources.	
/api/user-client/users/{user_id}	GET/PATCH	UserService	Requester must be self, platform authority, or authorized within active Client context.	SRS-FR-003
/api/user-client/roles	GET/POST	RoleService	Global role catalog access requires platform authority or approved administrator role.	SRS-FR-006
/api/user-client/clients/{client_id}/invitations	GET/POST	InvitationService	Invitation collection is filtered by client_id and active Client Administrator authority. Tenant scope must be derived from validated session context token; client-supplied query parameters or unverified headers are not authority sources.	SRS-FR-005, SRS-FR-012
/api/user-client/invitations/{invitation_id}/cancel	POST	InvitationService	Invitation must resolve to active_client_id before cancellation.	SRS-FR-005, SRS-VAL-007
/api/user-client/registrations/accept	POST	RegistrationService	Invitation token determines bound client_id; submitted user context cannot override it.	SRS-FR-005, SRS-VAL-008
/api/user-client/clients/{client_id}/client-memberships	GET/POST/PATCH	MembershipService	All membership queries and mutations are scoped by client_id and repository containment filters. active_client_id must be derived from validated session context and matched to the requested client_id.	SRS-FR-007, SRS-FR-012
/api/user-client/clients/{client_id}/project-memberships	GET/POST/PATCH	ProjectMembershipService	client_id and project containment must be validated before read or mutation. active_client_id must be derived from validated session context and matched to the requested client_id.	SRS-FR-008, SRS-VAL-005, SRS-VAL-011
/api/user-client/clients/{client_id}/reports/administrative	GET/POST	AdministrativeReportService	Report collection is scoped by client_id unless platform authority is explicitly present. Tenant scope must be derived from validated session context token; client-supplied query parameters or unverified headers are not authority sources.	SRS-FR-010, SRS-FR-011

7. DTO and Validation Envelope Contract

All request and response contracts must use typed Pydantic-style models inherited from the project engineering contract. Untyped dictionaries are not acceptable for core API contracts.

DTO	Required fields
ClientCreateRequest	code, legal_name, name, client_type_id, client_status_id, client_target_id, owner_user_id
ClientUpdateRequest	legal_name, name, description, client_type_id, client_status_id, client_target_id
UserCreateRequest	email, display_name, registration_type_id, user_status_id
InvitationCreateRequest	client_id, target_role_id, target_email, expires_at
RegistrationAcceptRequest	token, email, display_name
ClientMembershipRequest	client_id, user_id, role_id, membership_status_id
ProjectMembershipRequest	client_id, project_id, user_id, role_id, project_membership_status_id
ValidationErrorResponse	Explicit Pydantic model defined below; validation_id, message, entity_type, entity_id, client_id, trace_id with stable JSON serialization.
AdministrativeReportResponse	id, report_type, client_id, user_id, source_action, payload_json, generated_at, reviewed_at, archived_at

Explicit ValidationErrorResponse Pydantic contract:

```
from pydantic import BaseModel
from typing import Optional
```

```
class ValidationErrorResponse(BaseModel):
    validation_id: str # Must match stable SRS-VAL codes (e.g., "SRS-VAL-011") exactly
    message: str      # Prescriptive, human-readable business validation explanation
    entity_type: str  # Targeted database table identifier string matching Section 3
    entity_id: Optional[str] = None # Affected primary key tracking UUID or identifier string
    client_id: str     # Active client tenant tracking code enforcing containment context
    trace_id: str      # Secure alphanumeric platform correlation string for log inspection
```

Serialization rule: API validation failures must serialize as application/json using this field set exactly. Unknown fields are forbidden. validation_id must be one of the stable SRS-VAL identifiers defined by the frozen SRS and Section 9 of this ATS.

8. Application Service Contract

Service	Responsibility	Forbidden behavior
ClientService	Create, read, update, archive, restore, and validate Client records.	Bypass tenant checks or hard-delete Client records. Repository operations must not execute without the mandatory active_client_id containment filter where tenant-scoped data is read or mutated.
ClientLifecycleService	Apply Draft, Active, Suspended, and Archived transitions and cascade access freeze behavior.	Perform state transitions not listed in the SRS lifecycle contract.
UserService	Maintain global user identity records and profile state.	Grant tenant permissions directly from user existence. Repository operations must not execute without the mandatory active_client_id containment filter where tenant-scoped data is read or mutated.
RoleService	Maintain system-wide role catalog and role-type lookup binding.	Create per-client implicit roles outside approved role model.
InvitationService	Generate, cancel, expire, and verify invitation tokens.	Expose raw tokens after generation or allow token reuse across Clients. Repository operations must not execute without the mandatory active_client_id containment filter where tenant-scoped data is read or mutated.
RegistrationService	Accept invitation tokens and provision user/client membership state.	Register users into a Client without valid invitation and membership validation.
MembershipService	Create and update Client Membership records.	Create membership against archived Client or archived User. Repository operations must not execute without the mandatory active_client_id containment filter where tenant-scoped data is read or mutated.
ProjectMembershipService	Authorize project-level access within valid Client membership.	Create project membership without validating the parent Client membership. Repository operations must not execute without the mandatory active_client_id containment filter where tenant-scoped data is read or mutated.
TenantIsolationService	Centralize containment checks used by APIs and services. Repository Injection Directive: Every persistent database operation execution track must programmatically append a strict containment query filter (WHERE client_id = :active_client_id) directly inside the underlying repository layer. Service-layer authorization checks are secondary; repository query construction must natively guarantee absolute multi-tenant partition isolation and must raise SRS-VAL-011 on cross-tenant mismatch.	Allow ad hoc tenant checks scattered across controllers.
AdministrativeReportService	Persist immutable audit and administrative reports.	Mutate generated payload after initial persistence. Repository operations must not execute without the mandatory active_client_id containment filter where tenant-scoped data is read or mutated.

Repository Injection Directive: Every persistent database operation execution track must programmatically append a strict containment query filter (WHERE client_id = :active_client_id) directly inside the underlying repository layer for tenant-scoped tables. Service-layer authorization checks are

secondary; repository query construction must natively guarantee absolute multi-tenant partition isolation, throwing an SRS-VAL-011 exception if a cross-tenant data mismatch occurs. Global identity and global role lookups must still be joined through authorized Client Membership or platform authority before exposing data to callers.

9. Validation Implementation Contract

Validation ID	Technical enforcement
SRS-VAL-001	Required DTO fields are validated before service mutation and rechecked at persistence boundary.
SRS-VAL-002	client.code and user_account.email uniqueness checks execute before commit and are protected by database unique constraints.
SRS-VAL-003	Lifecycle transition service rejects any status change not in the lifecycle transition matrix.
SRS-VAL-004	Client activation verifies active lookup references for type, status, and target fields.
SRS-VAL-005	Project membership creation requires active Client Membership for the same client_id and user_id.
SRS-VAL-006	Client activation requires at least one active Client Administrator membership.
SRS-VAL-007	Invitation acceptance rejects expired, accepted, or cancelled tokens before registration.
SRS-VAL-008	Token integrity validation compares submitted token against stored token_hash.
SRS-VAL-009	Administrative report payload is immutable after persistence.
SRS-VAL-010	Membership records reject missing, archived, or unverified identity/workspace references.
SRS-VAL-011	Every project-context request must validate authenticated user, active client boundary, and project containment before mutation or read.

10. Lifecycle Transition Implementation Contract

Record type	Implementation owner	Blocking rule
Client / User	ClientLifecycleService / UserService	Archived records cannot become Active without restore validation; Active cannot revert to Draft after downstream relations exist.
Role	RoleService	Active roles mapped to live memberships cannot revert to Draft.
User Invitation	InvitationService	Expired invitations cannot return to Pending; Accepted invitations cannot be cancelled after registration completes.
Client Membership	MembershipService	Archived membership cannot become active without parent validation loop.
Project Membership	ProjectMembershipService	Project access cannot override Client containment.
Administrative Report	AdministrativeReportService	Archived reports cannot return to Reviewed without tracking signatures.

11. Asynchronous Administrative Processing Contract

Long-running administrative cascades must use native FastAPI BackgroundTasks only. External messaging frameworks or third-party task brokers are outside this contract layer.

Process	Trigger	Required behavior
Client suspension cascade	Client status changed to Suspended	Freeze subordinate memberships, project contexts, and non-platform user access paths.
Invitation expiration sweep	Scheduled or administrative request	Mark expired Pending invitations as Expired without creating membership records.
Multi-project membership update	Administrative bulk membership change	Apply project membership transitions through service validation and emit administrative reports.

Background Processing Failure Contract: If an asynchronous administrative background loop orchestrated via native FastAPI BackgroundTasks encounters a database deadlock, connection crash, or partial state cascade failure, it shall not fail silently. The orchestration loop must capture the exception, roll back all open database transactions, immediately log the complete structural failure trace payload to the unalterable administrative_report table with a state of Generated, and register an isolation fault category error message. Retry Policy: Background tasks are idempotent by process key and source action. Automatic retry is limited to one controlled retry for transient database connection failures only. Deadlocks, containment mismatches, validation failures, or cross-tenant errors must not retry automatically; they must stop, emit the administrative_report failure record, and expose a trace_id for operator review.

12. Frontend Screen Contract

Screen	Purpose	Required backend behavior
Client List	Browse authorized Client records and status.	Uses tenant-filtered client query.
Client Detail	View and update selected Client metadata and lifecycle.	Requires membership or platform authority.
User Registry	Browse global user identities where authorized.	Does not imply tenant access.

Invitation Management	Issue, view, cancel, and track invitations.	Requires active Client context and invitation status validation.
Client Membership Matrix	Manage User-to-Client memberships.	Requires active Client and administrator authority.
Project Membership Matrix	Manage User-to-Project memberships.	Requires active Client Membership and Project containment.
Role Catalog	Manage system-wide role catalog.	Prevents privilege escalation and tracks role lifecycle.
Administrative Reports	View immutable audit and compliance reports.	Filters by authorized Client context and report state.

13. Reporting and Audit Contract

- Every user provisioning, invitation generation, membership mutation, lifecycle transition, and isolation-breach failure must produce traceable audit evidence.
- Administrative reports must reference initiating user, target Client where applicable, target User where applicable, source action, source requirement or validation ID, timestamp, and structured payload.
- Report records are immutable after generation except for allowed Reviewed and Archived lifecycle markers.
- Generated reports must be readable by authorized administrators and exportable by project-level export mechanisms only after approval rules are satisfied.

14. Security and Authorization Contract

Rule	Implementation requirement
Authentication inheritance	Authentication, sessions, and identity establishment are inherited from the project-level security model.
Authorization evaluation	Application services evaluate platform authority, Client membership, Role assignment, and Project membership in that order.
Least privilege	Client Administrators manage only their Client boundary. Client Members read only assigned tenant and project records.
Platform authority	Platform Owner and System Administrator capabilities must be explicit and auditable, never implicit fallback behavior.
No direct permission mutation	Users do not receive permissions directly; authority flows through Role and Membership structures.

15. Detailed Field Matrix

The following field matrix is the deterministic database schema contract for the primary implementation tables. Generated PostgreSQL DDL and Pydantic model constraints must preserve these data types, nullability rules, uniqueness rules, foreign-key semantics, and immutability constraints unless a later approved ATS revision changes them explicitly.

Table	Field	Purpose	Technical Rule / Constraint
client	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
client	code	Stable unique code	VARCHAR(50), Not Null, Unique, Case-Insensitive Constraint, Immutable after activation
client	legal_name	Legal corporate name	VARCHAR(255), Not Null
client	name	Display name	VARCHAR(100), Not Null
client	description	Optional tracking remarks	TEXT, Nullable
client	client_type_id	Client classification reference	UUID, Not Null, FK to client_type.id, Block Cascade Delete
client	client_status_id	Client lifecycle status reference	UUID, Not Null, FK to client_status.id, Block Cascade Delete
client	client_target_id	Client operational target reference	UUID, Not Null, FK to client_target.id, Block Cascade Delete
client	owner_user_id	Responsible owner user reference	UUID, Not Null, FK to user_account.id, Block Cascade Delete
client	created_at	Creation timestamp	TIMESTAMP WITH TIME ZONE, Not Null, Generated Default
client	updated_at	Last update timestamp	TIMESTAMP WITH TIME ZONE, Not Null, Updated on mutation
client	archived_at	Soft archival timestamp	TIMESTAMP WITH TIME ZONE, Nullable
client_type	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
client_type	code	Lookup code	VARCHAR(50), Not Null, Unique
client_type	name	Lookup display name	VARCHAR(100), Not Null
client_type	description	Lookup description	TEXT, Nullable
client_type	is_active	Lookup availability flag	BOOLEAN, Not Null, Default TRUE
client_type	sort_order	Display order	INTEGER, Not Null, Default 0
client_status	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
client_status	code	Lifecycle status code	VARCHAR(50), Not Null, Unique
client_status	name	Lifecycle status name	VARCHAR(100), Not Null

client_status	description	Lifecycle status description	TEXT, Nullable
client_status	is_active	Lookup availability flag	BOOLEAN, Not Null, Default TRUE
client_status	sort_order	Display order	INTEGER, Not Null, Default 0
client_target	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
client_target	code	Target code	VARCHAR(50), Not Null, Unique
client_target	name	Target name	VARCHAR(100), Not Null
client_target	description	Target description	TEXT, Nullable
client_target	is_active	Lookup availability flag	BOOLEAN, Not Null, Default TRUE
client_target	sort_order	Display order	INTEGER, Not Null, Default 0
user_account	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
user_account	email	Unique global locator	VARCHAR(255), Not Null, Unique, Lowercase Index Constraint
user_account	display_name	Profile display name	VARCHAR(100), Not Null
user_account	description	Profile tracking notes	TEXT, Nullable
user_account	user_status_id	User lifecycle status reference	UUID, Not Null, FK to user_status.id, Block Cascade Delete
user_account	registration_type_id	Registration source reference	UUID, Not Null, FK to registration_type.id, Block Cascade Delete
user_account	created_at	Creation timestamp	TIMESTAMP WITH TIME ZONE, Not Null, Generated Default
user_account	updated_at	Last update timestamp	TIMESTAMP WITH TIME ZONE, Not Null, Updated on mutation
user_account	archived_at	Soft archival timestamp	TIMESTAMP WITH TIME ZONE, Nullable
user_status	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
user_status	code	User status code	VARCHAR(50), Not Null, Unique
user_status	name	User status name	VARCHAR(100), Not Null
user_status	description	User status description	TEXT, Nullable
user_status	is_active	Lookup availability flag	BOOLEAN, Not Null, Default TRUE
user_status	sort_order	Display order	INTEGER, Not Null, Default 0
registration_type	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
registration_type	code	Registration type code	VARCHAR(50), Not Null, Unique
registration_type	name	Registration type name	VARCHAR(100), Not Null
registration_type	description	Registration type description	TEXT, Nullable
registration_type	is_active	Lookup availability flag	BOOLEAN, Not Null, Default TRUE
registration_type	sort_order	Display order	INTEGER, Not Null, Default 0
role	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
role	code	Role code	VARCHAR(50), Not Null, Unique
role	name	Role name	VARCHAR(100), Not Null
role	description	Role description	TEXT, Nullable
role	role_type_id	Role type reference	UUID, Not Null, FK to role_type.id, Block Cascade Delete
role	is_active	Operational flag	BOOLEAN, Not Null, Default TRUE
role	created_at	Creation timestamp	TIMESTAMP WITH TIME ZONE, Not Null, Generated Default
role	updated_at	Last update timestamp	TIMESTAMP WITH TIME ZONE, Not Null, Updated on mutation
role	archived_at	Soft archival timestamp	TIMESTAMP WITH TIME ZONE, Nullable
role_type	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
role_type	code	Role type code	VARCHAR(50), Not Null, Unique
role_type	name	Role type name	VARCHAR(100), Not Null
role_type	description	Role type description	TEXT, Nullable
role_type	is_active	Lookup availability flag	BOOLEAN, Not Null, Default TRUE
role_type	sort_order	Display order	INTEGER, Not Null, Default 0
client_membership	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
client_membership	client_id	Parent Client context	UUID, Not Null, FK to client.id, Block Cascade Delete, Indexed
client_membership	user_id	Associated global user	UUID, Not Null, FK to user_account.id, Block Cascade Delete, Indexed
client_membership	role_id	Tenant-assigned role	UUID, Not Null, FK to role.id, Block Cascade Delete
client_membership	membership_status_id	Membership status reference	UUID, Not Null, FK to membership_status.id, Block Cascade Delete
client_membership	created_at	Creation timestamp	TIMESTAMP WITH TIME ZONE, Not Null, Generated Default
client_membership	updated_at	Last update timestamp	TIMESTAMP WITH TIME ZONE, Not Null, Updated on mutation
client_membership	archived_at	Soft archival timestamp	TIMESTAMP WITH TIME ZONE, Nullable
project_membership	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
project_membership	project_id	Target project context	UUID, Not Null, FK to project-level project abstraction, Indexed
project_membership	client_id	Parent Client isolation context	UUID, Not Null, FK to client.id, Block Cascade Delete, Indexed

project_membership	user_id	Target user reference	UUID, Not Null, FK to user_account.id, Block Cascade Delete, Indexed
project_membership	role_id	Project-assigned role	UUID, Not Null, FK to role.id, Block Cascade Delete
project_membership	project_membership_status_id	Project membership status reference	UUID, Not Null, FK to project_membership_status.id, Block Cascade Delete
project_membership	created_at	Creation timestamp	TIMESTAMP WITH TIME ZONE, Not Null, Generated Default
project_membership	updated_at	Last update timestamp	TIMESTAMP WITH TIME ZONE, Not Null, Updated on mutation
project_membership	archived_at	Soft archival timestamp	TIMESTAMP WITH TIME ZONE, Nullable
user_invitation	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
user_invitation	client_id	Bound Client tenant context	UUID, Not Null, FK to client.id, Block Cascade Delete, Indexed
user_invitation	issuer_user_id	Issuing administrator reference	UUID, Not Null, FK to user_account.id, Block Cascade Delete
user_invitation	target_role_id	Target role on acceptance	UUID, Not Null, FK to role.id, Block Cascade Delete
user_invitation	token_hash	Secure unique registration token	VARCHAR(64), Not Null, Unique
user_invitation	invitation_status_id	Invitation lifecycle status reference	UUID, Not Null, FK to invitation_status.id, Block Cascade Delete
user_invitation	expires_at	Expiration timestamp	TIMESTAMP WITH TIME ZONE, Not Null
user_invitation	accepted_at	Acceptance timestamp	TIMESTAMP WITH TIME ZONE, Nullable
user_invitation	cancelled_at	Cancellation timestamp	TIMESTAMP WITH TIME ZONE, Nullable
user_invitation	created_at	Creation timestamp	TIMESTAMP WITH TIME ZONE, Not Null, Generated Default
administrative_report	id	Primary Key	UUID, Not Null, Primary Key, Generated Default
administrative_report	report_type	Report classification	VARCHAR(50), Not Null, Indexed
administrative_report	client_id	Client context reference	UUID, Nullable, FK to client.id, Block Cascade Delete, Indexed
administrative_report	user_id	User context reference	UUID, Nullable, FK to user_account.id, Block Cascade Delete
administrative_report	source_action	Source action name	VARCHAR(100), Not Null
administrative_report	source_requirement_id	Source SRS requirement identifier	VARCHAR(50), Nullable
administrative_report	source_validation_id	Source SRS validation identifier	VARCHAR(50), Nullable
administrative_report	payload_json	Unalterable audit trail data	JSONB, Not Null, Completely Immutable after creation
administrative_report	generated_at	Generation timestamp	TIMESTAMP WITH TIME ZONE, Not Null, Generated Default
administrative_report	reviewed_at	Review timestamp	TIMESTAMP WITH TIME ZONE, Nullable
administrative_report	archived_at	Archive timestamp	TIMESTAMP WITH TIME ZONE, Nullable

16. Testing Contract

Test group	Required coverage
Unit tests	Services, validators, lifecycle transitions, token hashing, repository filters, and report immutability.
Functional tests	Client onboarding, invitation acceptance, membership assignment, project membership creation, suspension cascade, audit report extraction.
Negative tests	Cross-tenant access, expired token registration, archived user membership, project membership without client membership, invalid lifecycle transition.
Validation tests	Every SRS-VAL identifier must have at least one positive and one negative scenario where applicable.
UI tests	Screen-level behavior for client context, error display, membership management, and report views.
Generation validation	Generated artifacts must trace to SRS/ATS identifiers and must not modify files outside module scope.

17. Traceability Matrix

SRS requirement	ATS implementation areas
SRS-FR-001	client schema, ClientService, /clients endpoints, Client List/Detail screens, Section 15 field matrix
SRS-FR-002	TenantIsolationService, repository filters, authorization checks, SRS-VAL-011 tests, scoped collection endpoints
SRS-FR-003	user_account schema, UserService, /users endpoints, Section 15 field matrix
SRS-FR-004	lookup tables, activation validation, seed data
SRS-FR-005	user_invitation schema, InvitationService, RegistrationService, /registrations/accept, token_hash field contract
SRS-FR-006	role and role_type schemas, RoleService, Role Catalog screen
SRS-FR-007	client_membership schema, MembershipService, Client Membership Matrix, repository containment filters
SRS-FR-008	project_membership schema, ProjectMembershipService, Project Membership Matrix
SRS-FR-009	ClientLifecycleService, suspension cascade, background tasks, failure contract
SRS-FR-010	audit emission, administrative_report schema, payload_json immutability
SRS-FR-011	AdministrativeReportService, reports API, reports UI
SRS-FR-012	TenantIsolationService, service preconditions, repository containment filters

18. Seed Data and Lookup Values

Lookup	Approved values
client_status	Draft, Active, Suspended, Archived
user_status	Draft, Active, Suspended, Archived
role_status	Active, Archived
invitation_status	Pending, Accepted, Expired, Cancelled
membership_status	Active, Suspended, Archived
project_membership_status	Active, Disabled, Archived
administrative_report_status	Generated, Reviewed, Archived

19. Error and Gap Handling Contract

- Business logic gaps, structural field gaps, state sequence gaps, verification trace gaps, and code integration gaps block generation.
- The generator must produce a machine-readable gap report containing section, identifier, affected entity, severity, and recommended human clarification.
- Runtime validation errors must return stable validation_id values and must not expose sensitive tenant data from unauthorized scopes.
- ValidationErrorResponse serialization must use the typed Pydantic model defined in Section 7.

SpecificationGapManifest JSON contract:

```
{
  "gap_id": "string",
  "section": "string",
  "entity": "string",
  "severity": "Critical|Major|Minor|Editorial",
  "related_srs_id": "string|null",
  "related_ats_section": "string",
  "description": "string",
  "recommended_action": "string",
  "timestamp": "ISO-8601 timestamp",
  "trace_id": "string"
}
```

Serialization rule: Gap reports must be emitted as application/json and must be machine-readable without relying on prose parsing, strictly conforming to the SpecificationGapManifest JSON schema defined above. The generator must serialize this manifest file to the explicit target path: .cautomation_workspace/staging/{execution_id}/logs/specification_gaps.json. The generator must stop immediately after producing this manifest and must not create or modify implementation files for the unresolved scope.

20. Repository and File Generation Boundary

Generated implementation for this module must remain inside the approved User & Client Management module boundary. Shared project-level authentication, session, shell, export, and provider infrastructure must be referenced through approved abstractions rather than regenerated inside this module.

- Generated code must preserve project-level architecture and repository conventions.
- Whole-file generation is required unless a later project-level rule permits partial patch mode.
- Generated files must include manifest entries and traceability references to SRS and ATS sections.
- No Pipeline Management, AI Engine, Validation Engine, Apply Engine, or workflow execution artifacts may be created by this module ATS.
- Tenant-scoped repositories must enforce active_client_id containment at query-construction time before returning persistence results.

21. Minimum Generation Scope

A generated implementation is complete only when it delivers database schema, typed DTOs, repositories, services, REST endpoints, frontend administrative screens, validation rules, tenant-isolation enforcement, administrative reports, tests, and traceability reports matching the approved User & Client Management SRS and this ATS. If any required artifact cannot be generated without assumptions, the AI Engine must stop and emit a formal specification gap manifest.